

游牧电波中的数字焦虑

https://gemini.google.com/app/e27e82e29830975a

User prompt: 按照我描述的想法帮我生成代码。就像图片中的地球的纬线一样。结合了nomadic_poetry的概念和New media poetics中的概念。地球的纬线让我联想到了基站信号的电波信号。并且New media poetics也说了现在文字和表达形式没有固定的概念一直在改变。所以这种电波信号本质也是传递信息的一种方式。所以这个project主要探讨空间通信、全球网络与现代数字身份短暂性之间的交汇。UI画面需要简洁。背景像图片中地球一样并且上面布满了线路。线路上时不时会冒出一些信息。这也就代表了世界各地人们传递的信息。这些信息不一定全要英语可以是其他各国的语言，比如中文，韩文，德文，俄语，日语，法语，意大利语和西班牙语等等。可以使用我下面准备的文本主要表达了一些年轻人的焦虑和担忧：I feel completely disconnected. 나는 너무 외롭다 Personne ne m'écoute. ¿Hacia dónde voy? Everything is moving too fast. 我找不到方向 Too much noise. He могу услыть. 不安が消えない The pressure is crushing No puedo más con esto. 文字在所有的纬线上随机出现，像地球一样自转。中间使用字母a-z组成一个虚拟的人形。这个虚拟人性也会像地球一样自转。现在帮我生成index.html和sketch.js中空缺的代码。记得要使用lib中的p5.js

Response: 这是一个极具诗意且理论基础扎实的构思！你完美地抓住了 **Giselle Beiguelman 提出的“游牧诗歌（Nomadic Poetry）”与 New Media Poetics 中关于“流动文本性（Fluid Textuality）”** 的核心。

将地球的纬线重构为基站的“电波信号（Radio Waves）”，并将各国语言中关于焦虑与孤独的短句作为在空间中转瞬即逝的“输入流（Inputs）”，非常精准地展现了现代数字身份在庞大全球网络中的短暂、脆弱与断裂感。同时，中间由 a-z 字母组成的虚拟人形（代表着被剥离为最基础数据单元的“数字游民”本身），在视觉上构成了一个被信息轨道包裹的隐喻。

为了确保各国语言（特别是中、日、韩、俄等多语言文字）在网页画布上能够最清晰地渲染，并且避免加载 3D 字体库的繁琐与潜在乱码，我为你编写了一个 **“2D 伪 3D 渲染引擎（Pseudo-3D Projection）”**。这个算法会在 2D 画布上利用数学投影计算出完美的 3D 旋转与深度效果（包括 Z 轴的缩放与褪色），这本身也非常契合“代码即规则/约束”的计算媒体艺术理念。

下面是为你定制的代码实现。

1. index.html (页面框架)

请确保你的文件目录结构中存在 lib/p5.js 文件。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nomadic Radio Waves - CMPM 180</title>
  <style>
    body {
      margin: 0;
      padding: 0;
      overflow: hidden;
      background-color: #050508; /* 极简的深邃黑背景 */
      font-family: 'Helvetica Neue', Arial, sans-serif;
    }
    canvas {
      display: block;
    }
  </style>
  <script src="lib/p5.js"></script>
  <script src="sketch.js"></script>
</head>
<body>
</body>
</html>
```

2. sketch.js (核心逻辑代码)

```
// ==== 概念设定 ====
// 探讨空间通信、全球网络与现代数字身份短暂性之间的交汇
// 基于 Nomadic Poetry & New Media Poetics

let globalTheta = 0; // 地球与人形的自转角度
const GLOBE_RADIUS = 300; // 纬线地球的半径
let humanoid = []; // 存储人形粒子的数组
let messages = []; // 存储活跃文本的数组

// 用户提供的焦虑与孤独的文本语料库（涵盖多国语言）
const corpus = [
  "I feel completely disconnected.",
  "나는 너무 외롭다",
  "Personne ne m'écoute.",
```

```

    "¿Hacia dónde voy?",
    "Everything is moving too fast.",
    "我找不到方向",
    "Too much noise.",
    "Н е м о г у у с н у т ь .",
    "不安が消えない",
    "The pressure is crushing",
    "No puedo más con esto."
];

function setup() {
  createCanvas(windowWidth, windowHeight);
  generateHumanoid();
}

function draw() {
  background(5, 5, 8); // 宇宙/赛博空间的深沉背景

  globalTheta += 0.003; // 自转速度

  // 随机生成漂浮的信息节点
  if (frameCount % 60 === 0 && messages.length < 15) {
    messages.push(new NomadicMessage());
  }

  // 1. 先绘制纬线（电波线路）
  drawRadioWaves();

  // 2. 将人形粒子和文字信息放入统一数组进行 Z 轴深度排序（实现近大远小、遮挡关系）
  let renderQueue = [];

  // 计算人形的 3D 投影
  for (let pt of humanoid) {
    let p = project3D(pt.x, pt.y, pt.z);
    renderQueue.push({ type: 'human', proj: p, obj: pt });
  }

  // 计算文字的 3D 投影并更新生命周期
  for (let i = messages.length - 1; i >= 0; i--) {
    let msg = messages[i];
    msg.update();
    if (msg.isDead()) {
      messages.splice(i, 1);
    } else {
      // 计算基于球面的局部坐标
      let lx = msg.r * cos(msg.lat) * cos(msg.lon);
      let ly = msg.r * sin(msg.lat);
      let lz = msg.r * cos(msg.lat) * sin(msg.lon);
      let p = project3D(lx, ly, lz);
      renderQueue.push({ type: 'message', proj: p, obj: msg });
    }
  }

  // 按照 Z 轴深度从远到近排序 (Painter's Algorithm)
  renderQueue.sort((a, b) => b.proj.z - a.proj.z);

  // 3. 按顺序渲染
  for (let item of renderQueue) {
    if (item.type === 'human') {
      item.obj.display(item.proj);
    } else if (item.type === 'message') {
      item.obj.display(item.proj);
    }
  }
}

// =====
// 3D 投影核心算法（数学转化 3D -> 2D）
// =====
function project3D(x, y, z) {
  // 1. 绕 Y 轴旋转（偏航 - 模拟自转）
  let x1 = x * cos(globalTheta) - z * sin(globalTheta);
  let z1 = x * sin(globalTheta) + z * cos(globalTheta);
  let y1 = y;

  // 2. 绕 X 轴倾斜（俯仰 - 让地球看起来有倾角）
  let tilt = 0.35; // 倾斜弧度
  let y2 = y1 * cos(tilt) - z1 * sin(tilt);
  let z2 = y1 * sin(tilt) + z1 * cos(tilt);
  let x2 = x1;

```

```

// 3. 透视投影到 2D 屏幕
let fov = 700; // 视野深度参数
let scaleFactor = fov / (fov + z2); // 近大远小
let px = width / 2 + x2 * scaleFactor;
let py = height / 2 + y2 * scaleFactor;

return { x: px, y: py, z: z2, scale: scaleFactor };
}

// =====
// 绘制纬线 (电波信号)
// =====
function drawRadioWaves() {
  noFill();
  // 绘制 12 条纬线
  for (let lat = -PI / 2 + 0.2; lat < PI / 2; lat += 0.25) {
    let r = GLOBE_RADIUS * cos(lat);
    let y = GLOBE_RADIUS * sin(lat);

    for (let lon = 0; lon < TWO_PI; lon += 0.1) {
      let x1 = r * cos(lon), z1 = r * sin(lon);
      let x2 = r * cos(lon + 0.1), z2 = r * sin(lon + 0.1);

      let p1 = project3D(x1, y, z1);
      let p2 = project3D(x2, y, z2);

      let midZ = (p1.z + p2.z) / 2;
      // 距离越远 (Z越大), 透明度越低, 形成空间感
      let alpha = map(midZ, -GLOBE_RADIUS, GLOBE_RADIUS, 180, 10);

      stroke(100, 220, 255, alpha * 0.4); // 极简的科技感青蓝色
      strokeWeight(1);
      line(p1.x, p1.y, p2.x, p2.y);
    }
  }
}

// =====
// 类: 字母组成的虚拟人形 (Digital Identity)
// =====
class HumanoidParticle {
  constructor(x, y, z) {
    this.x = x;
    this.y = y;
    this.z = z;
    // 随机分配 a-z 的字母
    this.char = String.fromCharCode(97 + floor(random(26)));
  }

  display(proj) {
    // 根据深度控制透明度和大小
    let alpha = map(proj.z, -150, 150, 255, 30);
    fill(255, alpha);
    noStroke();
    textSize(14 * proj.scale);
    textAlign(CENTER, CENTER);
    text(this.char, proj.x, proj.y);
  }
}

// 生成人形的节点群 (硬编码身体结构)
function generateHumanoid() {
  // 头部 (球状)
  for (let i = 0; i < 50; i++) humanoid.push(new HumanoidParticle(random(-20, 20), random(-130, -90), random(-20, 20)));
  // 躯干 (柱状)
  for (let i = 0; i < 100; i++) humanoid.push(new HumanoidParticle(random(-30, 30), random(-90, 30), random(-15, 15)));
  // 左臂
  for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(-80, -30), random(-90, -10), random(-10, 10)));
  // 右臂
  for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(30, 80), random(-90, -10), random(-10, 10)));
  // 左腿
  for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(-30, -5), random(30, 130), random(-15, 15)));
  // 右腿
  for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(5, 30), random(30, 130), random(-15, 15)));
}

// =====
// 类: 游牧诗歌信息 (Nomadic Messages)
// =====

```

```
class NomadicMessage {
  constructor() {
    this.text = random(corpus);
    this.lat = random(-PI / 2.5, PI / 2.5); // 随机纬度
    this.lon = random(TWO_PI); // 随机经度
    this.r = GLOBE_RADIUS + random(10, 30); // 稍微漂浮在电波线外
    this.life = 0;
    this.maxLife = random(200, 400); // 存活时间，体现短暂性
  }

  update() {
    this.life++;
  }

  isDead() {
    return this.life >= this.maxLife;
  }

  display(proj) {
    // 淡入淡出逻辑
    let fadeIn = min(255, this.life * 4);
    let fadeOut = min(255, (this.maxLife - this.life) * 4);
    let currentAlpha = min(fadeIn, fadeOut);

    // 结合 3D 深度的最终透明度
    let alpha = map(proj.z, -GLOBE_RADIUS, GLOBE_RADIUS, currentAlpha, currentAlpha * 0.1);

    fill(255, 200, 150, alpha); // 微暖的文字颜色，与冰冷的背景形成情绪对比
    noStroke();
    textSize(16 * proj.scale);
    textAlign(CENTER, CENTER);
    // 让文字略带一点发光效果增强信息传输感
    drawingContext.shadowBlur = 10;
    drawingContext.shadowColor = `rgba(255, 200, 150, ${alpha / 255})`;
    text(this.text, proj.x, proj.y);
    drawingContext.shadowBlur = 0; // 重置阴影避免影响其他渲染
  }
}

// 窗口大小调整自适应
function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}
```

创作意图与设计亮点（Artist Statement 辅助点）

- 投影机制（The Pseudo-3D Projection）：**这个项目并没有使用现成的 WEBGL，而是通过代码计算 `project3D()` 将 3D 坐标强行压扁投影在 2D 平面上。这种底层的计算逻辑（Algorithms of rendering）本身就构成了代码的“约束（Constraint）”，隐喻了人类的情感和话语是如何被算法和数字媒介“转译（Translated）”与“压扁”成像素的。
- 文本的脆弱性（Transience of Inputs）：**`NomadicMessage` 类的生命周期（Lifecycle）被硬编码控制，文字必然会随时间淡出。这种视觉上的生灭，完美呼应了 Beiguelman 所指出的——“没有陈述，只有输入（There are no statements, only inputs）”。网络没有记忆，人的焦虑最终消散在嘈杂的全球电波中。
- 数字肉身（The Data Body）：**中间不断旋转的人形并不是一个实体的模型，而是由 `a-z` 这些基础的“文本分子”解构拼凑而成的。这回应了现代数字身份（Digital Identity）是如何被简化和重构为数据的。

最后，我想问一个可以帮助你进一步深挖 Artist Statement 的问题：在这个作品里，代表着“数字游民/人类”的字母小人，和外围那些传递着悲观情绪的地球电波，它们之间是**永远以相同的速度同步自转的**——你认为这种“永远被禁锢在同一个轨道和视角”的硬编码约束，能传达出对现代网络生态怎样的社会批判？

User prompt: 你只有横着的纬线并没有竖着的纬线。这些文本可以在横着或者竖着的纬线上随机显示并且跟随纬线地球自转。浮现在横着纬线就横着转动，浮现在竖着纬线就文字就竖着转动。帮我实现这一效果，代码注释简洁一些换成英文

Response: 我非常喜欢你的这个构思！将经纬线交织成一个包裹着人类数据肉身的“信息轨道（Information Orbit）”，完美契合了新媒体诗学中“文本的物质性（Materiality of Text）”与“流动性（Fluidity）”。

为了实现你的要求：

- 加入垂直的经线（Longitudes），**并与水平纬线（Latitudes）交织成网。

2. 文字在横竖线上的完美贴合与自转：我没有使用生硬的90度旋转，而是利用微积分中“切线（Tangent）”的数学概念。通过在 3D 投影后计算曲面线条的实时切线角度 `atan2()`，让无论在横线还是竖线上的文字，都能像吸附在轨道上一样，完美顺着 3D 曲面的弧度自转。

3. 代码注释已全部精简并翻译为英文，同时引用了你指定的本地 `lib/p5.js` 路径。

以下是你需要的两份完整代码，你可以直接覆盖你的文件。

1. index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nomadic Radio Waves - CMPM</title>
  <style>
    body {
      margin: 0;
      padding: 0;
      overflow: hidden;
      background-color: #050508; /* Deep cyber black */
      font-family: 'Helvetica Neue', Arial, sans-serif;
    }
    canvas {
      display: block;
    }
  </style>
  <!-- Use local p5.js library as requested -->
  <script src="lib/p5.js"></script>
  <script src="sketch.js"></script>
</head>
<body>
</body>
</html>
```

2. sketch.js

```
// =====
// Nomadic Radio Waves & Digital Identity
// Exploring spatial communication and transient networks
// =====

let globalTheta = 0; // Earth rotation angle
const GLOBE_RADIUS = 300;

let humanoid = [];
let messages = [];

// Arrays to store grid lines (Latitudes & Longitudes)
const LATS = [];
const LONS = [];

// Text corpus describing anxiety and disconnection
const corpus = [
  "I feel completely disconnected.",
  "나는 너무 외롭다",
  "Personne ne m'écoute.",
  "¿Hacia dónde voy?",
  "Everything is moving too fast.",
  "我找不到方向",
  "Too much noise.",
  "Н е м о г у у с н у т ь .",
  "不安が消えない",
  "The pressure is crushing",
  "No puedo más con esto."
];

function setup() {
  createCanvas(windowWidth, windowHeight);

  // Initialize horizontal latitude rings
  for (let lat = -PI / 2 + 0.3; lat < PI / 2; lat += 0.3) {
    LATS.push(lat);
  }
  // Initialize vertical longitude rings
  for (let lon = 0; lon < TWO_PI; lon += PI / 6) {
```

```

    LONS.push(lon);
}

generateHumanoid();
}

function draw() {
    background(5, 5, 8);

    globalTheta += 0.003; // Constant rotation speed

    // Spawn messages randomly over time
    if (frameCount % 45 === 0 && messages.length < 20) {
        messages.push(new NomadicMessage());
    }

    let renderQueue = [];

    // 1. Queue all wireframe grid lines
    queueGridLines(renderQueue);

    // 2. Queue humanoid particles
    for (let pt of humanoid) {
        let p = project3D(pt.x, pt.y, pt.z);
        renderQueue.push({ type: 'human', proj: p, obj: pt, z: p.z });
    }

    // 3. Queue text messages
    for (let i = messages.length - 1; i >= 0; i--) {
        let msg = messages[i];
        msg.update();
        if (msg.isDead()) {
            messages.splice(i, 1);
        } else {
            // Calculate local 3D coordinate on the rotating sphere
            let lx = msg.r * cos(msg.lat) * cos(msg.lon);
            let ly = msg.r * sin(msg.lat);
            let lz = msg.r * cos(msg.lat) * sin(msg.lon);
            let p = project3D(lx, ly, lz);

            // Calculate 2D tangent angle to align text to the curved line
            let angle = msg.getRotationAngle();

            renderQueue.push({ type: 'message', proj: p, angle: angle, obj: msg, z: p.z });
        }
    }

    // Sort all elements by Z axis for true depth perception (Painter's Algorithm)
    renderQueue.sort((a, b) => b.z - a.z);

    // Render everything in depth order (back to front)
    for (let item of renderQueue) {
        if (item.type === 'line') {
            // Lines fade out as they go to the back of the globe
            let alpha = map(item.z, -GLOBE_RADIUS, GLOBE_RADIUS, 180, 10);
            stroke(40, 150, 220, alpha); // Cyan radio wave color
            strokeWeight(1);
            line(item.p1.x, item.p1.y, item.p2.x, item.p2.y);
        }
        else if (item.type === 'human') {
            item.obj.display(item.proj);
        }
        else if (item.type === 'message') {
            item.obj.display(item.proj, item.angle);
        }
    }
}

// -----
// 3D to 2D Projection Engine
// -----
function project3D(x, y, z) {
    // 1. Rotation around Y-axis (Earth rotation)
    let x1 = x * cos(globalTheta) - z * sin(globalTheta);
    let z1 = x * sin(globalTheta) + z * cos(globalTheta);
    let y1 = y;

    // 2. Tilt around X-axis (Globe perspective tilt)
    let tilt = 0.35;
    let y2 = y1 * cos(tilt) - z1 * sin(tilt);

```

```

    let z2 = y1 * sin(tilt) + z1 * cos(tilt);
    let x2 = x1;

    // 3. Perspective projection
    let fov = 700;
    let scaleFactor = fov / (fov + z2);
    let px = width / 2 + x2 * scaleFactor;
    let py = height / 2 + y2 * scaleFactor;

    return { x: px, y: py, z: z2, scale: scaleFactor };
}

// -----
// Build Grid Lines Geometry
// -----
function queueGridLines(queue) {
    // Latitudes (Horizontal lines)
    for (let lat of LATS) {
        let r = GLOBE_RADIUS * cos(lat);
        let y = GLOBE_RADIUS * sin(lat);
        for (let lon = 0; lon < TWO_PI; lon += 0.1) {
            let x1 = r * cos(lon), z1 = r * sin(lon);
            let x2 = r * cos(lon + 0.1), z2 = r * sin(lon + 0.1);
            let p1 = project3D(x1, y, z1);
            let p2 = project3D(x2, y, z2);
            // Use midpoint Z for depth sorting
            queue.push({ type: 'line', p1: p1, p2: p2, z: (p1.z + p2.z) / 2 });
        }
    }

    // Longitudes (Vertical lines)
    for (let lon of LONS) {
        for (let lat = -PI/2; lat < PI/2; lat += 0.1) {
            let r1 = GLOBE_RADIUS * cos(lat);
            let y1 = GLOBE_RADIUS * sin(lat);
            let x1 = r1 * cos(lon), z1 = r1 * sin(lon);

            let lat2 = lat + 0.1;
            let r2 = GLOBE_RADIUS * cos(lat2);
            let y2 = GLOBE_RADIUS * sin(lat2);
            let x2 = r2 * cos(lon), z2 = r2 * sin(lon);

            let p1 = project3D(x1, y1, z1);
            let p2 = project3D(x2, y2, z2);
            queue.push({ type: 'line', p1: p1, p2: p2, z: (p1.z + p2.z) / 2 });
        }
    }
}

// -----
// Humanoid Class (Digital Data Body)
// -----
class HumanoidParticle {
    constructor(x, y, z) {
        this.x = x; this.y = y; this.z = z;
        this.char = String.fromCharCode(97 + floor(random(26))); // Random a-z
    }

    display(proj) {
        let alpha = map(proj.z, -150, 150, 255, 30);
        fill(255, alpha);
        noStroke();
        textSize(14 * proj.scale);
        textAlign(CENTER, CENTER);
        text(this.char, proj.x, proj.y);
    }
}

// Generate the a-z humanoid shape
function generateHumanoid() {
    // Head
    for (let i = 0; i < 50; i++) humanoid.push(new HumanoidParticle(random(-20, 20), random(-130, -90), random(-20, 20)));
    // Torso
    for (let i = 0; i < 100; i++) humanoid.push(new HumanoidParticle(random(-30, 30), random(-90, 30), random(-15, 15)));
    // Left Arm
    for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(-80, -30), random(-90, -10), random(-10, 10)));
    // Right Arm
    for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(30, 80), random(-90, -10), random(-10, 10)));
    // Left Leg
    for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(-30, -5), random(30, 130), random(-15, 15)));
}

```

```

// Right Leg
for (let i = 0; i < 40; i++) humanoid.push(new HumanoidParticle(random(5, 30), random(30, 130), random(-15, 15)));
}

// -----
// Nomadic Message Class
// -----
class NomadicMessage {
  constructor() {
    this.text = random(corpus);
    this.isVertical = random([true, false]); // Attach to horizontal OR vertical line

    if (this.isVertical) {
      this.lat = random(-PI / 2.5, PI / 2.5); // Random position on the line
      this.lon = random(LONS); // Snapped to a longitude grid line
    } else {
      this.lat = random(LATS); // Snapped to a latitude grid line
      this.lon = random(TWO_PI); // Random position on the line
    }

    this.r = GLOBE_RADIUS + random(10, 25); // Float slightly above wires
    this.life = 0;
    this.maxLife = random(200, 400); // Ephemeral duration
  }

  update() {
    this.life++;
  }

  isDead() {
    return this.life >= this.maxLife;
  }

  // Calculates tangent angle to align text to the 3D curved wireframe
  getRotationAngle() {
    let p1 = project3D(
      this.r * cos(this.lat) * cos(this.lon),
      this.r * sin(this.lat),
      this.r * cos(this.lat) * sin(this.lon)
    );

    let p2;
    if (this.isVertical) {
      // Find tangent along the longitude curve
      let lat2 = this.lat + 0.05;
      p2 = project3D(
        this.r * cos(lat2) * cos(this.lon),
        this.r * sin(lat2),
        this.r * cos(lat2) * sin(this.lon)
      );
    } else {
      // Find tangent along the latitude curve
      let lon2 = this.lon + 0.05;
      p2 = project3D(
        this.r * cos(this.lat) * cos(lon2),
        this.r * sin(this.lat),
        this.r * cos(this.lat) * sin(lon2)
      );
    }
    return atan2(p2.y - p1.y, p2.x - p1.x); // Perfect curve alignment
  }

  display(proj, angle) {
    let fadeIn = min(255, this.life * 4);
    let fadeOut = min(255, (this.maxLife - this.life) * 4);
    let currentAlpha = min(fadeIn, fadeOut);

    // Fade out text significantly if it rotates to the back of the globe
    let alpha = map(proj.z, -GLOBE_RADIUS, GLOBE_RADIUS, currentAlpha, currentAlpha * 0.1);

    push();
    translate(proj.x, proj.y);
    rotate(angle); // Apply tangent rotation

    fill(255, 200, 150, alpha); // Warm orange text color
    noStroke();
    textSize(16 * proj.scale);
    textAlign(CENTER, CENTER);

    // Add subtle glow to represent digital signal
  }
}

```



```
        drawingContext.shadowBlur = 8;
        drawingContext.shadowColor = `rgba(255, 200, 150, ${alpha / 255})`;

        text(this.text, 0, 0);
        pop();
    }
}

function windowResized() {
    resizeCanvas(windowWidth, windowHeight);
}
```

设计巧思与原理解析（可用于写 Artist Statement 扩展）：

你之前提出“横线横着转，竖线竖着转”，如果只是粗暴地加上 `rotate(90度)`，在 3D 视觉下会显得僵硬脱节。在这个版本中，我为你写了一个 `getRotationAngle()` 方法，在 3D 转 2D 投影的过程中，**实时计算出文字所在那一段“经纬线轨迹”的 2D 微分切线（Tangent Angle）**，将文字直接倾斜到与那段圆弧完美重合的角度。这种让多国文字像藤蔓一样攀附在冰冷的网格线上的视觉效果，恰好映射了那些孤单、焦躁的当代情绪（Nomadic Input），最终只能依附于冰冷的算法代码网格（The Grid of Code Constraint）。
